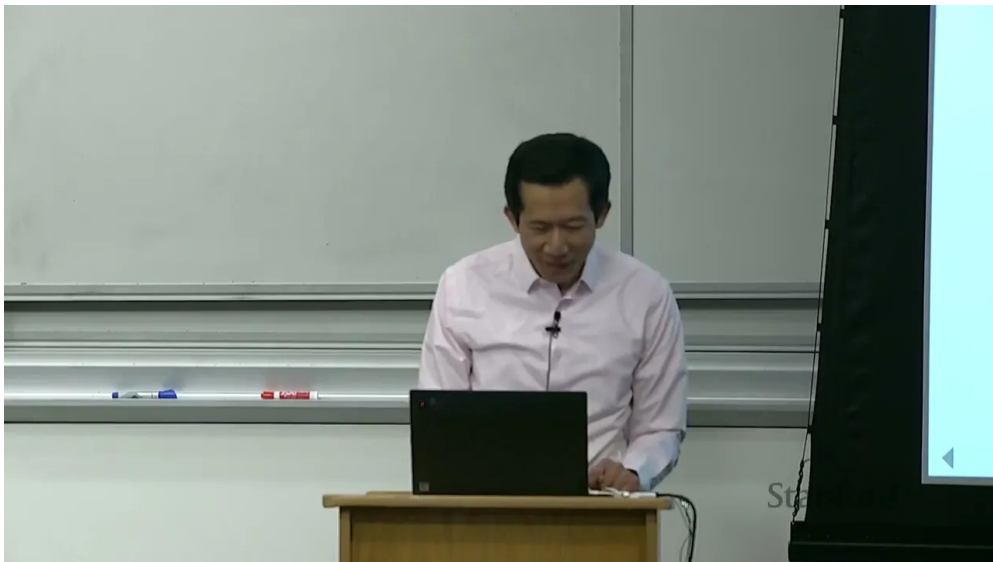


课程笔记

CS336 Lecture 8: Distributed Training Across Multiple GPUs

基于课程字幕与代码脚本整理

基于公开视频字幕整理



视频作者/频道: Stanford CS336

发布日期: 未提供

视频时长: 约 1:15:00

视频链接: 未填写

目录

1 引言：把计算尽量放到离数据最近的地方	2
2 集合通信：分布式编程的基本积木	2
2.1 术语	2
2.2 常见原语	3
2.3 为什么这些原语重要	3
3 硬件与软件栈	3
3.1 旧式路径和现代路径	3
3.2 NCCL 与 torch.distributed	4
3.3 带宽基准的意义	4
4 数据并行：按 batch 切分	4
4.1 工作方式	4
4.2 为什么 loss 可能不同，但参数仍一致	5
4.3 你可以把它看成什么	5
5 张量并行：按宽度切分	5
5.1 工作方式	5
5.2 和数据并行的差别	6
5.3 为什么讲里只演示 forward	6
6 流水线并行：按深度切分	6
6.1 工作方式	6
6.2 naive 版本的问题	6
7 更广义的视角：切分维度与代价	7
7.1 三种代价	7
7.2 为什么这套结构不会消失	7
8 JAX、TPU 与更高层抽象	7
8.1 PyTorch 的位置	7
8.2 JAX 的位置	8
8.3 为什么课程里仍然选 PyTorch	8
9 本讲总结	8

1 引言：把计算尽量放到离数据最近的地方

这节课延续上一讲关于单 GPU 内部并行的讨论，主题转向 **多 GPU / 多节点** 场景。核心问题并没有变：**计算单元离输入输出总是很远**，而性能优化的关键，就是想办法减少数据搬运，把工作组织成更高的算术强度。

本讲的统一主题

1. 上一讲在单 GPU 内通过 fusion 和 tiling 减少对 HBM 的访问。
2. 这一讲在多 GPU / 多节点内通过 replication、sharding 和通信原语减少跨设备传输。
3. 所有方法的目标都一样：让计算尽量“就地完成”，避免把数据来回搬。

可以把整套系统抽象成一个从 **小而快到 大而慢** 的层次结构：

层次	典型位置	特征
L1 cache / shared memory	单 GPU 内部	极快，但容量很小
HBM / DRAM	单 GPU 内部	容量大很多，但仍然比片上存储慢
NVLink	同一节点内多 GPU	GPU 之间直接互联
NVSwitch	多节点多 GPU	更大规模互联与路由

表 1: 从小快到大慢的层次视角

一句话记忆

如果数据已经在本地缓存里，计算就很快；如果数据在别的 GPU 上，通信就会成为瓶颈。多 GPU 训练的核心任务，就是重新安排计算和存储，使这个瓶颈尽量晚出现。

2 集合通信：分布式编程的基本积木

在分布式训练里，最基础的抽象不是“GPU”本身，而是 **collective operations**。它们描述的是一组 rank 之间的标准通信模式，比手工管理点对点通信更稳定，也更容易优化。

2.1 术语

- **World size**: 参与通信的设备数量。
- **Rank**: 某一台设备的编号，通常从 0 开始。

2.2 常见原语

原语	直觉	用途
broadcast	一份数据发给所有 rank	模型参数同步、配置下发
scatter	一份输入切成多份分给不同 rank	数据切片
gather	不同 rank 的数据收集到一个 rank	汇总输出
reduce	对多个值做 sum/min/max 等归约	梯度聚合、统计量合并
all-gather	每个 rank 都拿到完整拼接结果	张量并行中的激活恢复
reduce-scatter	先归约，再切片分发	all-reduce 的一半
all-reduce	所有 rank 最终都拿到归约结果	DDP 梯度同步

表 2: 本讲使用的集合通信原语

三个容易混淆的点

1. **reduce** 不是任意操作，而是需要可结合、可交换的归约，如 sum、min、max、average。
2. **all** 的意思是“所有 rank 都得到结果”。
3. **all-reduce** 可以拆成 **reduce-scatter** + **all-gather**，这也是很多底层实现的思路。

2.3 为什么这些原语重要

这些操作并不只是“通信工具”，它们本身就是分布式训练算法的语言。你可以把很多并行策略理解成：先决定 **哪一部分数据或模型是本地的**，再决定 **哪一部分必须通过 collective 交换**。

3 硬件与软件栈

3.1 旧式路径和现代路径

早期常见的结构是：GPU 通过 PCIe 连接到 CPU，再通过主机网络卡和 Ethernet 与别的节点通信。这个路径能工作，但链路长、拷贝多、开销大。

路径	典型特征	问题
GPU → CPU → Ethernet	通用性强，历史上很常见	数据搬运太长，延迟和带宽都不理想
GPU → NVLink → GPU	同节点内直接互联	适合高频率的大张量交换
GPU → NVSwitch → GPU	跨节点直接互联	适合更大规模的训练集群

表 3: 通信路径的演化

一个现实约束

PCIe、Ethernet、NVLink、NVSwitch 的性能都在变，但层次关系不会变：**越接近算子执行位置，越快；越依赖主机中转，越慢。**

3.2 NCCL 与 torch.distributed

NVIDIA 提供了 **NCCL** 来把高层 collective 原语翻译成低层通信包和 CUDA kernel。它会先识别硬件拓扑，再选择合适的传输路径，最后完成数据发送与接收。

在 PyTorch 里，`torch.distributed` 提供了更高层的接口：

- `init_process_group` 初始化分布式环境；
- `all_reduce` / `reduce_scatter_tensor` / `all_gather_into_tensor` 等接口直接对应 collective；
- `send` / `recv` 用于点对点通信；
- `barrier` 用于显式同步；
- `destroy_process_group` 用于清理进程组。

分布式编程里的同步点

像 `all_reduce` 这样的操作本身就是同步点。某个 rank 少做一次，其他 rank 就可能一直等下去，程序表现为“挂住”。

3.3 带宽基准的意义

讲里专门演示了如何用简单的脚本测量 NCCL 带宽。这个动作的意义不只是“看数字”，而是要确认：

- 硬件拓扑是否符合预期；
- collective 的实现是否真的利用到了高速链路；
- 训练瓶颈究竟在计算还是在通信。

4 数据并行：按 batch 切分

数据并行是最直接的思路：**每个 rank 持有完整模型，但只处理一部分 batch**。前向和反向都在本地完成，然后在反向结束后对梯度做 all-reduce，同步所有 worker 的参数更新。

4.1 工作方式

1. 将 batch 沿着 batch 维度切成多份。
2. 每个 rank 拿到自己的本地数据片段。
3. 每个 rank 维护一份完整的模型副本。

4. 本地完成前向、损失和反向。
5. 对每一层的 `param.grad` 做 all-reduce 平均。
6. 继续各自的 optimizer step。

DDP 的本质

数据并行里，**参数是复制的，数据是切分的，梯度是同步的**。从优化器视角看，很多时候就像做了 world size 份 SGD，但梯度在每步都被强制对齐了。

4.2 为什么 loss 可能不同，但参数仍一致

每个 rank 看到的是不同的本地 batch，所以局部 loss 不必相同。真正保证训练一致性的，是反向结束后对梯度做了同步平均。只要初始化相同、同步相同，参数更新就会保持一致。

一个需要注意的边界条件

如果模型里有依赖整个 batch 统计量的层，比如某些 batch norm 变体，就需要额外考虑跨 rank 的统计同步。讲里提到，在语言模型场景里常见的是 layer norm，所以这个问题一般不会像视觉模型里那么棘手。

4.3 你可以把它看成什么

数据并行其实是在“省通信”和“省复杂度”之间做折中。它的好处是实现简单，缺点是模型仍然必须完整地复制到每个 GPU 上，所以当模型本身太大时，它就不够用了。

5 张量并行：按宽度切分

张量并行把模型沿着 **hidden dimension / width** 切开。每个 rank 只保存每层的一部分参数，但它要频繁和其他 rank 交换激活值，才能把完整的前向算下去。

5.1 工作方式

1. 模型参数按列或按隐藏维度切分。
2. 每个 rank 只持有每层参数的一部分。
3. 本地先算出局部激活。
4. 用 all-gather 把局部激活拼回完整表示。
5. 进入下一层，重复上述过程。

张量并行的代价

张量并行适合“模型太宽，装不下”的场景，但它的代价是 **激活通信很多**。所以这类方案通常非常依赖高速互连，否则就会被通信拖死。

5.2 和数据并行的差别

数据并行主要交换 **梯度**；张量并行主要交换 **激活**。前者在反向阶段集中通信，后者几乎每一层前向都在通信。

策略	切分对象	主要通信内容
数据并行	batch 维度	梯度 all-reduce
张量并行	hidden / width 维度	激活 all-gather / 相关变体
流水线并行	layer / depth 维度	相邻 stage 间 send / recv

表 4: 三种并行策略的对照

5.3 为什么讲里只演示 forward

讲里的 toy example 只把前向过程写出来，因为 backward 的 bookkeeping 更复杂。核心思想已经足够清楚：**每个 rank 只算自己那一片，但为了继续下一层，必须把表示恢复完整。**

6 流水线并行：按深度切分

流水线并行把模型按 **layer / depth** 切开。每个 rank 负责若干层，数据则在各 rank 间像流水线一样流动。

6.1 工作方式

1. 把 batch 切成多个 microbatch。
2. rank 0 持有输入数据，后续 rank 持有中间表示。
3. 每个 rank 先 recv 上一个 rank 的输出。
4. 本地执行自己负责的层。
5. 用 send 把结果传给下一 rank。

流水线并行的直觉

你可以把它看成一条装配线：每个工位只负责一段工序，数据逐段往前流。microbatch 的作用，是让工位尽量保持忙碌，减少“空等”的 bubble。

6.2 naive 版本的问题

讲里明确指出，最原始的流水线实现还有不少问题：

- send / recv 是同步的，不能重叠通信与计算；
- forward 和 backward 的调度还没有展开；
- 真正高性能的实现需要更复杂的时序安排。

流水线并行的关键瓶颈

如果不做异步和交叠，流水线并行会产生 bubble，GPU 不是在算就是在等。因此实际系统里最重要的优化，往往是把“等待通信”尽量藏进“计算中”。

7 更广义的视角：切分维度与代价

把三种策略放在一起看，就能发现它们只是对同一问题的不同切分：

- **数据并行**：切 batch；
- **张量并行**：切 width；
- **流水线并行**：切 depth；
- 讲里还提到，类似的思想也能扩展到 sequence / context length。

7.1 三种代价

你最终总会在下面三者之间取舍：

1. **重算**：省内存，但会多做算力；
2. **存内存**：省算力，但会占空间；
3. **跨 GPU 通信**：能让模型装得下，但会增加链路开销。

讲里的总括

很多并行方法，本质上都在回答同一个问题：到底是把中间结果重新算一遍，还是把它存在更远的地方，再付出通信代价取回来。

7.2 为什么这套结构不会消失

即便硬件继续变快，模型也会继续变大、变宽、变深。只要模型规模逼近硬件极限，层次化的存储和通信结构就还会存在。也就是说，并行和通信优化不是临时补丁，而是深度学习系统的常态。

8 JAX、TPU 与更高层抽象

讲里顺带对比了 PyTorch 和 JAX/TPU 生态。

8.1 PyTorch 的位置

PyTorch 的优势是灵活、透明，适合把底层发生了什么讲清楚。讲里用它来实现 data / tensor / pipeline 并行，就是为了让学生看见原语级别的操作。

8.2 JAX 的位置

JAX 的思路更偏声明式：你定义模型，再声明如何切分，编译器去负责剩余部分。这样做的结果是，很多分布式细节可以被隐藏起来，代码也更短。

JAX 视角的好处

1. 更高层的 sharding 描述；
2. 编译器自动生成通信和调度；
3. 对大规模 TPU 集群更友好。

8.3 为什么课程里仍然选 PyTorch

因为这门课想让你理解“系统到底怎么转起来”的内部机制。如果直接用更高层的封装，很多通信和同步细节会被隐藏，你很难知道性能瓶颈和正确性约束从哪里来。

9 本讲总结

- 多 GPU 训练的本质问题，仍然是 **如何减少数据搬运**。
- 集合通信是分布式训练的基础积木，尤其是 all-reduce、all-gather、reduce-scatter。
- 数据并行切 batch，张量并行切 width，流水线并行切 depth。
- 数据并行主要同步梯度，张量并行主要同步激活，流水线并行主要在相邻 stage 之间传递中间结果。
- 高性能系统总是在 **重算、内存和通信**三者之间找平衡。
- PyTorch 适合展示底层细节，JAX 适合展示更高层的 sharding 抽象。

最后一句话

硬件会持续演进，但“计算离数据很远”这件事不会消失。只要模型继续长大，分层存储、分层通信、分层并行就仍然是深度学习系统设计的核心。